

可靠传输

用有限状态驯服丢失、损坏与乱序

408 · NET-03

1 本册定位

本 Topic 回答：下层只提供可能损坏、丢失、重复和乱序的交付时，通信端点怎样共同制造“正确且不重复”的可靠服务？

本册拥有 sequence number、ACK、timer、retransmission、sender/receiver window、Stop-and-Wait、GBN、SR，以及窗口利用率与序号空间约束。

本册建立通用可靠传输机制，不把它等同于 TCP。TCP 如何把 byte stream、连接状态与这些机制组合，见传输层与 TCP；cwnd 不属于本册，见拥塞控制。

2 独立阅读词典：可靠到底保证什么

这里的“可靠”不是承诺链路永不出错，而是让接收应用看到的交付满足题目指定的顺序、无重复和完整性要求。下表固定本册所有流程图的词义：

术语	本册中的可操作定义	不要混成什么
sequence number / ACK	序号给数据定位；ACK 反馈“某序号已收到”或“此前连续前缀已收到”	ACK 不是对方一定收到的全局证明
cumulative ACK	一个值确认此前连续数据；individual ACK 只确认指定数据	不能把两种 ACK 记法混用
timer / retransmission	等待确认的本地时钟；到期后再次发送同一数据	超时不等于已证明原数据丢失
window	当前允许发送或接受的序号范围，决定能有多少数据在途	不是文件的固定分块大小
GBN / SR	GBN 失序少缓存、缺口后批量回退；SR 缓存失序、只重传缺失帧	不是两种不同的可靠目标
duplicate suppression	用序号和接收状态保证重传副本不再次向上交付	不是阻止网络中出现副本
reliable delivery	让应用看到题设要求的顺序、无重复和完整性	不是承诺链路永不出错

因此，“超时”只能推出一个可执行动作：重传或进入题设指定的失败分支；它不能单独证明原数据丢失，因为 ACK 也可能丢失，或报文仍在传播。

3 根本问题：沉默不能说明发生了什么

发送一个数据单元后没有收到反馈，至少存在三种解释：

1. 数据丢失；
2. 数据到达，但 ACK 丢失；
3. 数据或 ACK 仍在路上。

端点没有全局观察者，不能直接知道网络内部发生了什么。可靠传输只能通过有限的报文和本地时间推断状态：

Error detection → Identity → Feedback → Timeout → Retransmission → Duplicate suppression

可靠不是某一个字段的性质，而是发送方与接收方状态机形成的闭环。

4 五个最小机制为什么缺一不可

4.1 Error detection：先知道“这个副本不能信”

校验和或 CRC 让接收方检测部分传输错误。检测失败通常触发丢弃或 NAK，但它不能恢复原数据。

4.2 Sequence number：给传输实例一个可比较身份

重传会产生多个内容相同的副本。若没有序号，接收方无法判断“这是新数据还是旧数据重传”。序号还为顺序和累计确认提供坐标系。

4.3 ACK：把接收端知识反馈给发送端

ACK 的语义必须先写清：确认单个序号、累计确认到某个序号，还是声明下一个期望序号。不同教材或协议可能采用不同记法，不能只凭 ACK(n) 猜。

4.4 Timer：把无限等待变成可执行分支

发送方无法等到“确定丢失”，只能在等待超过阈值后推定本次尝试失败。Timer 太短会产生伪重传，太长会延迟恢复。

4.5 Retransmission + duplicate suppression

超时重传解决可能丢失；序号与接收状态消除重传造成的重复交付。二者共同把“至少一次尝试”收缩为“至多一次向上交付”。

5 Stop-and-Wait：最小闭环

发送方状态：

```
1 Ready (n)
2 -> send packet (n), start timer
3 -> WaitACK (n)
4   -> valid ACK (n): stop timer, n toggles, Ready (next)
```

```
5    -> timeout: resend packet(n), restart timer
```

接收方状态:

```
1 Expect (n)
2 -> valid packet(n): deliver once, send ACK(n), Expect(next)
3 -> duplicate/out-of-order: do not deliver, repeat appropriate ACK
4 -> corrupted: discard or send feedback according to protocol
```

1-bit 序号足以支持 Stop-and-Wait, 因为任意时刻只需区分当前新 packet 与上一个 packet 的迟到/重传副本。

5.1 正确性不变量

- 发送方收到足以推进状态的确认前, 不复用当前序号;
- 接收方只把当前期望序号向上交付一次;
- ACK 丢失可以造成重复传输, 但不能造成重复交付。

5.2 性能失败

若数据发送时间为 T_D 、单向传播时延为 T_P , 并把 RTT 简化为 $2T_P$, 忽略 ACK 发送与处理:

$$U \approx \frac{T_D}{T_D + RTT}.$$

当 $RTT \gg T_D$, 链路大部分时间空闲。可靠性已经成立, 但吞吐能力没有被利用, 于是需要流水线。

6 Sliding Window: 把等待时间变成在途数据

窗口不是“批量发送”的口号, 而是对允许未确认序号集合的约束:

Past ACKed | Sent unACKed | Usable unsent | Not yet allowed

ACK 到达使左边界右移, 释放新的序号。窗口同时承担:

- correctness: 限制哪些序号当前合法;
- flow of work: 允许多少数据在反馈返回前继续发送;
- bounded state: 限定发送方、接收方和缓冲区必须维护多少状态。

若窗口含 W 个等长 frame, ACK 发送时间为 T_{ACK} , 理想无差错利用率近似:

$$U = \min\left(1, \frac{WT_D}{T_D + RTT + T_{ACK}}\right).$$

公式必须从时间线生成。题目忽略 ACK 发送时间时才去掉 T_{ACK} 。

7 GBN: 用接收端简单换取批量回退

GBN 的接收窗口通常为 1: 只接收当前期望 frame, 失序 frame 不缓存。发送方允许多个未确认 frame 在途, 并使用累计确认。

7.1 生命周期

```

1 sender sends base ... nextseq-1
2 -> receiver accepts only expected frame
3 -> cumulative ACK advances base
4 -> if oldest unACKed timer expires
5 -> resend base and every later outstanding frame

```

7.2 为什么只需一个主计时器

累计确认使最老未确认 frame 成为窗口能否前进的阻塞点。其超时后，后续在途 frame 即使曾正确到达也可能因接收方不缓存而需要一起重传。

7.3 序号空间约束

若序号字段为 k bit，在 408 常用模型中：

$$W_T \leq 2^k - 1, \quad W_R = 1.$$

必须保留至少一个序号不在当前发送窗口中，才能让新一轮窗口与旧副本保持可区分。

8 SR：用更多状态换取精准恢复

SR 的接收方可以缓存窗口内失序 frame，并逐个确认；发送方通常为每个未确认 frame 维护独立计时状态。丢一个 frame 时只重传所缺 frame。

8.1 生命周期

```

1 frame n arrives within receive window
2 -> if first valid copy: buffer and ACK n
3 -> if n equals receive base: deliver n and consecutive buffered frames
4 -> slide receive window over delivered prefix

```

接收方可以“已收到但暂不能交付”。这说明 reception state 与 delivery state 不是同一件事。

8.2 序号复用为什么会产生歧义

序号是有限循环空间。若发送窗口和接收窗口覆盖范围过大，一个迟到的旧 frame 可能落入新一轮接收窗口，并被误认为新 frame。

通用不歧义条件是：

$$W_T + W_R \leq 2^k.$$

常用对称窗口下：

$$W_T = W_R \leq 2^{k-1}.$$

这个约束不是为了“方便计算”，而是为了让接收方仅凭有限序号和当前窗口就能判定身份。

9 GBN 与 SR 的真正权衡

维度	GBN	SR
接收失序 frame	丢弃	缓存
ACK	通常累计	通常逐 frame
超时恢复	从缺口开始批量重传	只重传具体缺失 frame
接收端状态	小	大
发送端 timer	通常最老 frame 一个主 timer	每个未确认 frame 独立状态
高误码链路代价	重传浪费较大	状态和实现复杂度较大

SR 不是无条件更好。链路很可靠、窗口较小或接收资源受限时，GBN 的简单性可能更值钱。

10 408 状态机覆盖：发送端与接收端双账本

任何 ARQ 流程都用同一张事件表推进：

事件	报文/字段	发送端状态变化	接收端状态变化
发送新 frame	seq、payload、checksum	占用窗口槽；启动相应 timer	无
正确到达	seq 在/不在接收窗口	等待 ACK	接收/缓存/交付；产生 ACK
损坏或失序	checksum/seq 不满足	等待 ACK 或超时	丢弃、重复 ACK 或 NAK（按题设）
ACK 到达	ACK 语义必须先声明	标记确认；连续前缀满足才滑窗	无
timer 到期	对应 seq 或 oldest unACKed	Stop-and-Wait/GBN/SR 按各自策略重传	无

10.1 Stop-and-Wait 的四个异常分支

必须分别检查 data loss、data corruption、ACK loss、ACK corruption/delay。发送方超时后都可能重传；接收方用期望序号保证重复副本只回 ACK、不重复 deliver。停止条件不是“某个副本到达”，而是 sender 获得足以安全复用序号的反馈。

10.2 GBN: base、nextseqnum 与唯一主 timer

发送端允许

$$base \leq seq < base + W_T.$$

ACK 的累计语义只推进连续确认前缀。收到旧 ACK 不回退 base；最老未确认 frame 超时则重传区间 $[base, nextseqnum)$ 。接收端窗口为 1，失序 frame 不缓存，并重复报告当前连续前缀。每个事件后检查

$$base \leq nextseqnum \leq base + W_T.$$

10.3 SR: 接收、缓存、交付是三种状态

发送端逐 frame 维护 ACK 与 timer; 接收端对窗口内首个有效副本缓存并选择确认。只有 receive base 已到达时, 才把它和随后连续缓存项一起 deliver 并滑动窗口。迟到但已落在旧窗口的副本可重复确认却不得再交付; 窗口外未来序号按题设丢弃。序号空间必须满足 $W_T + W_R \leq 2^k$, 否则仅凭 seq 无法区分旧副本与新循环。

停止类比

本册的 GBN/SR 是通用 ARQ 模型。TCP 可以调用累计 ACK、失序缓存和选择确认等机制, 但不能被整体贴成 GBN 或 SR; TCP 的 byte sequence、connection 和 flow control 由 NET06 Own。

11 计算与反例生成: 从反馈周期到序号空间

11.1 一个完整周期不能漏掉 ACK 与处理阶段

设数据帧发送时延为 T_f 、ACK 发送时延为 T_a 、单向传播时延为 τ 、接收处理时延为 T_p 。停止等待从开始发送一帧到可以安全发送下一帧的周期为

$$T_{cycle} = T_f + \tau + T_p + T_a + \tau = T_f + 2\tau + T_p + T_a, \quad U_{SW} = \frac{T_f}{T_{cycle}}.$$

只有题设明确忽略 ACK 与处理开销时, 才化为

$$U_{SW} = \frac{1}{1 + 2a}, \quad a = \frac{\tau}{T_f}.$$

这说明提高链路速率会缩短 T_f , 却不缩短 τ ; 若窗口仍为 1, a 反而变大, 利用率可能下降。超时阈值应覆盖正常反馈周期并留有余量: 过短产生伪重传, 过长延迟真实丢失恢复。链路层 RTT 较稳定时可用固定余量, 端到端 TCP 的动态 RTT 估计由 NET06 拥有。

11.2 流水线满载阈值与序号位数反推

发送窗口为 W 时, 理想无差错利用率为

$$U(W) = \min\left(1, \frac{WT_f}{T_f + 2\tau + T_p + T_a}\right).$$

因此满载所需的最小整数窗口为

$$W_{sat} = \left\lceil \frac{T_f + 2\tau + T_p + T_a}{T_f} \right\rceil.$$

忽略 T_p, T_a 时, $W_{sat} = \lceil 1 + 2a \rceil$ 。再把物理阈值接到协议上限:

$$\text{GBN: } 2^n - 1 \geq W_{sat} \implies n \geq \lceil \log_2(W_{sat} + 1) \rceil,$$

$$\text{SR, } W_T = W_R: 2^{n-1} \geq W_{sat} \implies n \geq 1 + \lceil \log_2 W_{sat} \rceil.$$

顺序必须是“先由链路时间线求窗口需求, 再由协议不歧义约束求序号位数”, 不能直接把某个 2^n 公式当利用率。

11.3 ACK 丢失为什么在 GBN 与 SR 中代价不同

GBN 的 ACK 是连续前缀摘要。若 ACK 3 丢失而 ACK 5 到达, ACK 5 已包含“0-5 都按序收到”的知识, 发送方可以直接推进到 6; 较大的累计 ACK 覆盖较小 ACK。SR 的 ACK 通常是逐帧状态: ACK 3 丢失不能由 ACK 4 或 ACK 5 自动证明 frame 3 已收到, 因此 frame 3 的 timer 仍可能到期并产生一次冗

余重传。接收方收到已交付窗口之前的旧 frame 时，必须重新发送该 frame 的 ACK，否则发送方可能永远无法关闭对应 timer。

这不是“SR 的 ACK 更差”，而是反馈信息结构不同：

GBN：压缩连续前缀，恢复粒度粗	SR：表达任意接收模式，状态与定时器更多
------------------	----------------------

11.4 发送窗口与接收窗口不要求同步

发送窗口在最早未确认项获得足够确认时滑动；接收窗口在其左端起的连续帧已经收到并可交付时滑动。两个条件分别发生在 ACK 回程与 data 到达事件上，所以窗口位置天然可以不同步。SR 的发送窗口内部还可能出现“前项未确认、后项已确认”的洞；窗口左边界不能越过洞，但相应的后项无需重传。

接收 frame 时要分三类：

- 在当前接收窗口内：首次有效副本则缓存并 ACK；只有补齐左端连续前缀时才按序交付并滑窗；
- 在接收窗口之前：数据已处理，丢弃副本但重复 ACK；
- 在接收窗口之后：尚不合法，按题设丢弃。

11.5 有限序号的隐藏前提：旧副本寿命必须有界

Stop-and-Wait 用 1 bit 序号，只能区分“当前期望的新帧”和“上一帧的重传”。它隐含旧副本不会跨越多个完整序号循环后仍从信道中出现。短而有界的链路模型通常满足这一前提；若网络允许报文任意长时间滞留，仅靠 0/1 交替不能防止极迟旧副本被当成新数据。窗口不重叠约束解决一个反馈周期内的新旧歧义，报文最大寿命与更大序号空间则解决跨周期身份复用风险。

12 来源覆盖附录：NET-03 逐 Source 核销

Source	本册承接节点	核销结论
datalink/stop-and-wait	最小闭环/反馈周期	四异常分支、1-bit 序号、timeout 权衡、含 ACK/处理时延的利用率与满载反推已展开
datalink/gbn	累积确认/批量回退	$W_R = 1, W_T \leq 2^n - 1$ 、唯一主 timer、乱序丢弃、ACK 覆盖与重传区间已展开
datalink/sr	逐帧确认/选择重传	缓存与按序交付、逐 frame timer、三类接收位置、 $W_T + W_R \leq 2^n$ 与旧帧反例已展开
datalink/sliding-window-compare	统一窗口模型	四区域、两端异步滑动、分段利用率、三协议上限/代价/适用条件及传输层边界已展开

13 概念边界

概念 A	≠ 概念 B	真正区别与题目信号	混淆后果
Detection	≠ Recovery	检错指出异常；反馈/超时/重传恢复	认为 CRC 自带重传
Receive	≠ Deliver	frame 可到达并缓存，但未必按序交给上层	SR 状态推演错误
ACK loss	≠ Data loss	前者接收方已有数据；后者没有	漏掉重复抑制分支

概念 A	≠ 概念 B	真正区别与题目信号	混淆后果
Window size	≠ Sequence space	窗口是当前合法集合；序号空间是循环身份全集	GBN/SR 最大窗口算错
Reliability	≠ Flow control	前者保护数据交付；后者保护接收缓冲	把 <code>rwnd</code> 当作可靠性条件
Flow control	≠ Congestion control	前者看 receiver；后者看 network path	把 <code>rwnd</code> 、 <code>cwnd</code> 合并成一
Timeout	≠ Proof of loss	超时是基于本地时钟的推断，不是全局事实	无法解释伪重传

14 Verification：状态题怎样提前发现错误

每一步检查四个集合：

1. 已确认/已交付的连续前缀；
2. 已发送未确认集合；
3. 已接收但未交付缓存；
4. 当前窗口允许的新序号。

再检查三个不变量：

- 同一数据不能向上交付两次；
- 窗口只跨过连续满足条件的前缀；
- 当前接受区间不能与仍可能出现的旧副本产生身份歧义。

15 做题调用协议

1. 先写 ACK 语义和接收方是否缓存失序 frame；
2. 画 sender/receiver 两条时间线，标出发送、到达、ACK、超时；
3. 每个事件后更新 `base`、`nextseq`、`buffer`、`timer`；
4. 窗口题先判断 GBN 还是 SR，再写对应不歧义条件；
5. 利用率题从一个 frame 的反馈周期推导，不先套公式；
6. 有误码/丢包时同时算重传数据量和状态成本；
7. 最后用“是否可能重复交付”攻击结果。

16 贯穿母例：ACK 丢失为什么不会破坏可靠性

Stop-and-Wait 中，发送方发送 frame 0；接收方正确接收、交付并回 ACK 0，但 ACK 丢失：

```

1 sender remains WaitACK(0)
2 -> timer expires
3 -> sender retransmits frame 0
4 -> receiver is already Expect(1)
5 -> recognizes duplicate 0, does not deliver again
6 -> repeats ACK 0
7 -> sender advances to frame 1

```

重传保证 progress，序号与接收状态保证 safety。可靠性的两面分别是：

eventually make progress

never deliver the wrong instance twice.

17 高频 First Divergence

- 看见丢包直接重传：没有说明谁、凭什么知道丢失；
- ACK 到达就把窗口跳到该序号之后：没有确认 ACK 是累计还是选择确认；
- SR 收到失序 frame 立即交付：混淆 buffer 与 in-order delivery；
- GBN 只重传超时的一个 frame：把恢复策略套成 SR；
- 只背 $2^k - 1$ 或 2^{k-1} ：没有从旧/新副本不歧义生成约束；
- 利用率超过 1：没有执行饱和截断或时间线有重复计数。

18 一页压缩与复原问题

Detect → Number → Acknowledge → Time out → Retransmit → Suppress duplicate

1. 为什么“没收到 ACK”不能等价为“数据丢了”？
2. Stop-and-Wait 为什么 1-bit 序号足够？
3. 窗口怎样把传播等待转化为在途数据？
4. GBN 和 SR 各把复杂度放在哪一端？
5. SR 的序号空间约束怎样从迟到旧 frame 推出？

19 来源与校正说明

- 归档笔记《链路层-流量控制与可靠传输》《公式汇总》提供 GBN/SR 考点和旧计算模型；
- 本册把可靠传输提升为跨层可复用机制，避免被旧目录永久绑定在链路层；
- ACK 记号在不同教材中可能不同，正文要求先声明语义，不把单一记法冒充协议普遍事实。